

Pointers in C

Direct Memory Manipulation & Logic

Definitions • Dereferencing • Pointer Arithmetic

1. What is a Pointer?

A **pointer** is a variable that stores the memory address of another variable instead of storing a value directly.

Syntax:

```
data_type *pointer_name;
```

```
int a = 10;  
int *p = &a; // p stores address of a
```

Dereferencing: Using the `*` operator to access the value at the address stored in the pointer.

2. Essential Operators

Operator	Name	Description
<code>&</code>	Address-of	Returns the memory address of a variable.
<code>*</code>	Dereference	Used to access the value at the address held by a pointer.

```
printf("%d", a); // Output: 10
printf("%d", *p); // Output: 10 (Accesses value at address)
```

3. Pointer Arithmetic

Arithmetic operations like + and - behave differently with pointers. When you add 1 to an `int` pointer, it moves forward by the size of an integer (usually 4 bytes).

```
int *ptr = &a;  
ptr++; // Moves to next memory block based on data type
```

Types of Pointers:

- **Null Pointer:** `int *p = NULL;` (Points to nothing).
- **Void Pointer:** `void *p;` (Generic pointer type).
- **Wild Pointer:** Uninitialized pointer (dangerous).

4. Pointers with Arrays

An array name acts as a constant pointer to its first element. This is why pointers are extremely efficient for traversing arrays.

```
int arr[3] = {10, 20, 30};  
int *ptr = arr; // Same as &arr[0]  
  
for(int i=0; i<3; i++) {  
    printf("%d ", *(ptr + i)); // Accessing using pointer  
}
```

5. Common Pitfalls

- **Uninitialized Pointers:** Using a pointer before assigning an address (leads to Segmentation Fault).
- **Dangling Pointers:** A pointer pointing to a memory location that has been deleted or freed.
- **Size Mismatch:** Assigning a float address to an `int*` without proper casting.

Pro Tip: Always initialize pointers to NULL if you don't have an address to assign immediately.



Module Complete

Pointers are the "Power Tool" of C. Mastering them allows for dynamic memory management and hardware-level programming.

*** Final Page ***